Static Analysis for Go Channels

Aleksei Kosarev

Study project · University of Freiburg
Supervisor: Prof. Peter Thiemann
13 October 2026

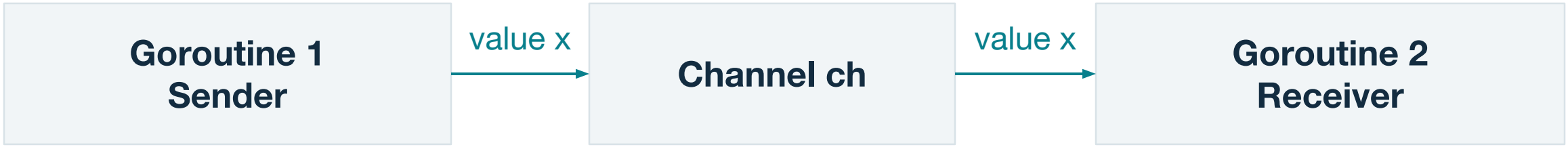
Goroutines and Channels

Goroutines

Lightweight concurrent execution
Started with the go keyword

Channels

Values passed between goroutines
Send: ch <- x Receive: <-ch



Unbuffered: send and receive must meet

The sender waits if no receiver is ready

What Is a Deadlock?

Goroutines are permanently blocked and cannot make progress

Correct communication

G1: [!, ?]
G2: [?, !]

Both communications can complete

Deadlock

G1: [!, ?]
G2: [!, ?]

Both try to send first
Neither can proceed

! = send ? = receive The order of operations matters

Why do we need Static Analysis?

Runtime detection

- Requires execution
- Observes the behaviour of that run
- May miss partial deadlocks

Static analysis

- Inspects the program without running it
- Explores abstract possibilities
- Warns about potential deadlocks

Detect potential deadlocks before running the program

Research Question and Scope

Can we detect potential deadlocks in Go programs without executing them?

Current scope

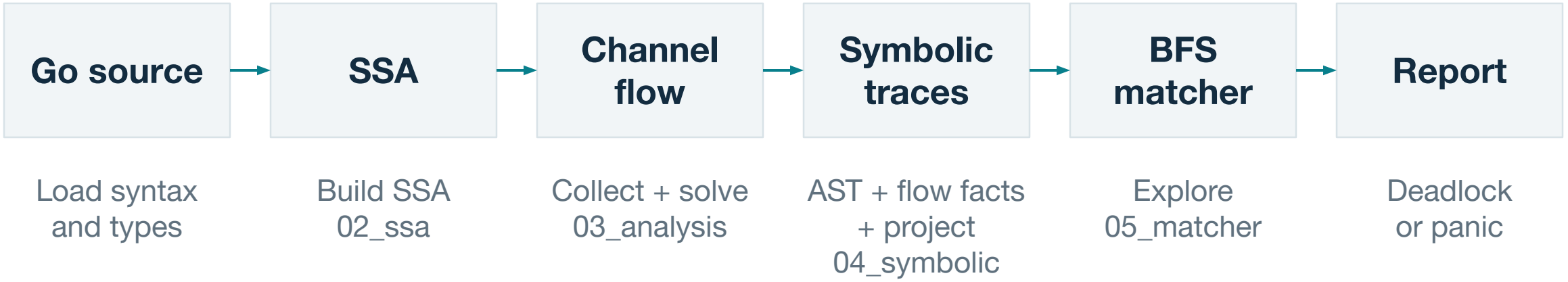
- Unbuffered channels
- Send, receive and close
- If/else, loops and channel range
- Channel flow through function calls

Result

- Potential deadlock warnings
- Invalid channel operation warnings

No warning is not a safety proof

Analysis Pipeline



Trace extraction uses the AST as well as SSA-linked flow facts

Channel Flow: Which channel reaches a goroutine?

```
func sender(ch chan int) {
    ch <- 10
}

func main() {
      a := make(chan int)
      go sender(a)
      <-a
}
```

Allocation site (ID, Position, Type)

The source location of
make(chan int)

ContextValue (SSA Value, Goroutine)

SSA value: main.sender.ch

Goroutine context: main.go:7

State[main.main.t0@main] = { allocation site of a }
State[main.sender.ch@main.go:7] = { allocation site of a }

Channel Flow: Core constraint generation

Source construct	Generated constraint
v := make(chan T)	p ∈ State[v]
dst := src	State[src] ⊆ State[dst]
phi := ϕ(..., input, ...)	State[input] ⊆ State[phi]
call func(..., arg, ...)	State[arg] ⊆ State[param]
return value	State[value] ⊆ State[func.return]
result := call func(...)	State[func.return] ⊆ State[result]

p is the allocation site program point

- Sets only grow by union
- Repeat until no new facts can be added
(Fixed Point)

Symbolic traces: Representation

Symbol	Meaning	Example in Go code
!	Send (write) to a channel	ch <- 5
?	Receive (read) from a channel	v := <-ch
X	Close a channel	close(ch)
loop(n, [...])	A loop with a known number of iterations	for i := 0; i < n; i++ { ... }
loop(*, [...])	A loop with an unknown number of iterations	for { ... }
if(cond, [...], [...])	A conditional branch with then/else traces	if cond { ... } else { ... }
?*	Receive in a range loop (read until channel is closed)	for v := range ch { ... }

Symbolic traces: Example

Sender

```
func send(ch chan int) {
    for i := 0; i < 2; i++ {
        ch <- i
    }
    close(ch)
}
```

[loop(2, [!]), X]

Receiver

```
func recv(ch chan int) {
    for range ch {
    }
}
```

[?*]

Channel operations are converted into symbolic traces while preserving their order and structure

Symbolic traces: Projection

Original trace in one goroutine

[?(A), !(B), X(B)]

Projection	Remaining trace
Channel A	[?]
Channel B	[!, X]

The dependency “receive A before sending B” disappears

BFS matcher: Small-step Operational Semantics

$$S = (\Gamma, \text{closed}) \quad \Gamma(g) = \text{remaining trace of goroutine } g$$

Fix one projected channel c. ! = Write(c) ? = Read(c) X = Close(c)

$$\frac{\Gamma(g_1) = ! \cdot \tau_1 \quad \Gamma(g_2) = ? \cdot \tau_2 \quad g_1 \neq g_2 \quad \text{closed=false}}{(\Gamma, \text{false}) \rightarrow (\Gamma[g_1 \mapsto \tau_1, g_2 \mapsto \tau_2], \text{false})} \text{ (Rendezvous)}$$

$$\frac{\Gamma(g) = X \cdot \tau \quad \text{closed=false}}{(\Gamma, \text{false}) \rightarrow (\Gamma[g \mapsto \tau], \text{true})} \text{ (Close)}$$

τ is a remaining trace · concatenates traces Γ[g ↦ τ] updates one trace

BFS matcher: Example

Step	Sender	Receiver	Channel
Initial	[loop(2, [!]), X]	[?*]	Open
Loop-N	[!, loop(1, [!]), X]	[?*]	Open
First rendezvous	[loop(1, [!]), X]	[?*]	Open
Loop-One	[!, X]	[?*]	Open
Second rendezvous	[X]	[?*]	Open
Close	[ ]	[?*]	Closed
Range finishes	[ ]	[ ]	Closed

Without close(ch), the receiver would remain blocked at [?*]

BFS matcher: Breadth-first search and termination

Search strategy

- Explore states in breadth-first order
- Use state hashing to avoid repeated exploration

Exploration budget

- Stop after more than 1,000 distinct states
- Reaching the limit produces a Deadlock warning

Reaching the state limit does not prove a deadlock

Deadlock detection

- No outgoing transition exists from current state
- At least one goroutine has unfinished operations with channel
- All goroutines with unfinished traces are blocked

Testing and evaluation

20 unit-test scenarios

8 solver / 5 projection / 7 matcher

Benchmark suite — 36 Go programs	Programs
gopherlyzer / dingo-hunter	6 / 9
gcatch / literature / custom	1 / 16 / 4
Total: 23 Deadlock + 13 Safe	36

Expected labels are stored in oracle.csv

Results on 36 programs

Oracle label	Warning	No warning
Deadlock	19 TP	4 FN
Safe	5 FP	8 TN

79.17%

Precision = 19 / (19 + 5)

82.61%

Recall = 19 / (19 + 4)

Panic and budget warnings count as positive

Why the analyser misses some deadlocks

Lost control flow

```
if skip {
    return
}
ch <- 20
```

The trace may still contain [!]

Lost cross-channel order

G1: [!(A), ?(B)]

G2: [!(B), ?(A)]

- Each channel alone has a pair
- Together, both goroutines send first

APPENDIX

BFS matcher: Small-step Operational Semantics

All operations below refer to the same projected channel c

$$\frac{\Gamma(g) = ? \cdot \tau \quad \text{closed=true}}{(\Gamma, \text{true}) \to (\Gamma[g \mapsto \tau], \text{true})} \text{ (Read-Closed)}$$

$$\frac{\Gamma(g) = \text{Op}(m,c) \cdot \tau \ \wedge \ m \in \{\text{Write}, \text{Close}\} \quad \text{closed=true}}{(\Gamma, \text{true}) \to \text{Panic}} \text{ (Panic)}$$

A closed channel permits a receive, but not another send or close

BFS matcher: Small-step Operational Semantics

Let If = If(cond, α, β), L = Loop(*, β), and d ∈ {false, true}

$$\frac{\Gamma(g) = \mathrm{If} \cdot \tau}{(\Gamma,d) \to (\Gamma[g \mapsto \alpha \cdot \tau],d)}\ \textbf{(If-Then)} \qquad \frac{\Gamma(g) = \mathrm{If} \cdot \tau}{(\Gamma,d) \to (\Gamma[g \mapsto \beta \cdot \tau],d)}\ \textbf{(If-Else)}$$

$$\frac{\Gamma(g) = \mathrm{L} \cdot \tau}{(\Gamma,d) \to (\Gamma[g \mapsto \tau],d)}\ \textbf{(Loop-Exit)} \qquad \frac{\Gamma(g) = \mathrm{L} \cdot \tau}{(\Gamma,d) \to (\Gamma[g \mapsto \beta \cdot \mathrm{L} \cdot \tau],d)}\ \textbf{(Loop-Step)}$$

Each pair has the same premise and two separate successor states

BFS matcher: Small-step Operational Semantics

Let L(N) = Loop(N, β), with integer $0 \le N \le 100$ and $d \in \{\text{false}, \text{true}\}$

$$\frac{\Gamma(g) = L(0) \cdot \tau}{(\Gamma, d) \to (\Gamma[g \mapsto \tau], d)} \quad \textbf{(Loop-Zero)}$$

$$\frac{\Gamma(g) = L(1) \cdot \tau}{(\Gamma, d) \to (\Gamma[g \mapsto \beta \cdot \tau], d)} \quad \textbf{(Loop-One)}$$

$$\frac{\Gamma(g) = L(N) \cdot \tau \quad \wedge \quad 1 < N \le 100}{(\Gamma, d) \to (\Gamma[g \mapsto \beta \cdot L(N-1) \cdot \tau], d)} \quad \textbf{(Loop-N)}$$

These rules use the extracted bound. The extractor only applies a literal heuristic.

BFS matcher: Small-step Operational Semantics

Let R = Range(c, β), a range receives until the channel closes

$$\frac{\Gamma(g_1) = ! \cdot \tau_1 \quad \Gamma(g_2) = R \cdot \tau_2 \quad g_1 \neq g_2 \quad \text{closed=false}}{(\Gamma, \text{false}) \to (\Gamma[g_1 \mapsto \tau_1, g_2 \mapsto \beta \cdot R \cdot \tau_2], \text{false})} \text{(Range-Step)}$$

$$\frac{\Gamma(g) = R \cdot \tau \quad \text{closed=true}}{(\Gamma, \text{true}) \to (\Gamma[g \mapsto \tau], \text{true})} \text{(Range-End)}$$

On an open channel, range cannot choose to exit without a receive

Current implementation boundaries

Area	Current boundary
Communication	No select or buffer state
Channel flow	No closure capture
Program entry	main only, no package-level channel initialisation
Control flow	Early return, break and continue are not supported
Loops and calls	Limited support for loop iterations and recursive calls
Goroutine identity	Source go location, not per runtime instance
Exploration	Per-channel search, 1,000-state budget